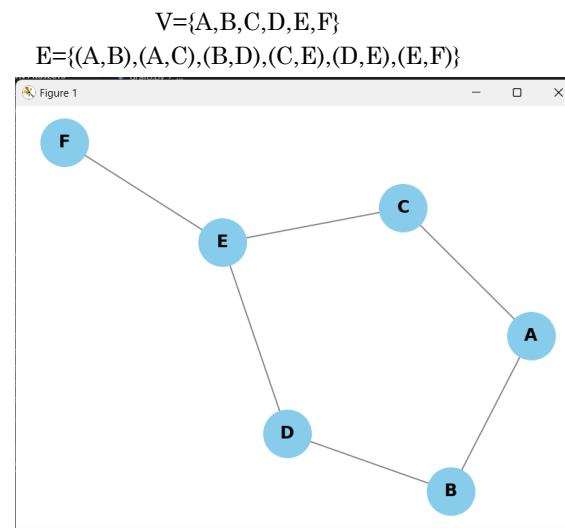


1. Dibujar el siguiente grafo:

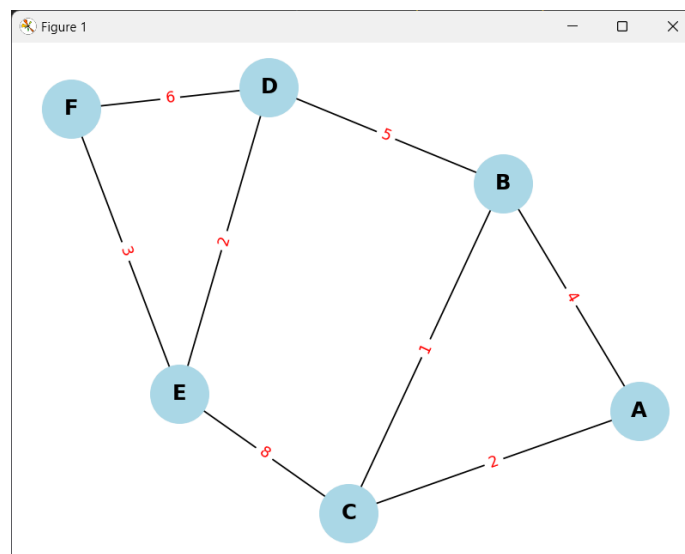


2. Representar el grafo anterior usando listas de adyacencia.

$A \rightarrow B, C$
 $B \rightarrow A, D$
 $C \rightarrow A, E$
 $D \rightarrow B, E$
 $E \rightarrow C, D, F$
 $F \rightarrow E$

3. Dibujar el siguiente grafo ponderado:

$V = \{A, B, C, D, E, F\}$
 $E = \{(A, B), (A, C), (B, C), (B, D), (C, E), (D, E), (D, F), (E, F)\}$
 $w(A, B) = 4, w(A, C) = 2, w(B, C) = 1, w(B, D) = 5, w(C, E) = 8, w(D, E) = 2, w(D, F) = 6, w(E, F) = 3$



4. Representar el grafo anterior usando una matriz de adyacencia.

	A	B	C	D	E	F
A	0	4	2	0	0	0

B	4	0	1	5	0	0
C	2	1	0	0	8	0
D	0	5	0	0	2	6
E	0	0	8	2	0	3
F	0	0	0	6	3	0

5. Utilizando las librerías networkx (<https://networkx.org/documentation/stable/index.html>) y matplotlib dibujar el grafo anterior.

```
import networkx as nx
import matplotlib.pyplot as plt

# Crear grafo no dirigido
G = nx.Graph()

# Añadir aristas con pesos
aristas = [
    ("A", "B", 4),
    ("A", "C", 2),
    ("B", "C", 1),
    ("B", "D", 5),
    ("C", "E", 8),
    ("D", "E", 2),
    ("D", "F", 6),
    ("E", "F", 3)
]
G.add_weighted_edges_from(aristas)
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_color="lightblue", node_size=1500,
font_size=14, font_weight="bold")
labels = nx.get_edge_attributes(G, "weight")
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels, font_color="red")
plt.title("Grafo ponderado con pesos  $w(u,v)$ ")
plt.show()
```

6. Repasando recursividad, implementar los siguientes algoritmos recursivos:
- Calcular el factorial de un número $n!$, usando recursividad.

```
import sys
def factorial(n:int)->int:
    #caso base
    if n <=1 :
        return 1
    #caso recursivo
    else:
        return n*factorial(n-1)
def main():
```

```

#analisis de los parámetros de entrada
if len(sys.argv) == 2:
    n = sys.argv[1]
    try:
        numero = int(n)
        resultado=factorial( numero)
        print(f"El resultado es: {resultado}")

    except ValueError:
        print(f"Error: '{n}' no es un número entero válido.")

else:
    print("Uso python factorial.py <n>")
if __name__ == "__main__":
    main()

```

- Implementar una función que de forma recursiva indique si una cadena es palíndromo o no.

```

import sys
def palindromo(palabra:str)->bool:
    #caso base
    if len(palabra)==1 or len(palabra)==2 and palabra[0]==palabra[1] :
        return True
    #caso recursivo
    else:
        #se compara la primera y la última
        if(palabra[0]==palabra[-1:]):
            #se evalua pero quitando la primera y la última
            return palindromo(palabra[1:-1])
        else:
            return False
def main():
    #analisis de los parámetros de entrada
    if len(sys.argv) == 2:
        palabra = sys.argv[1]
        resultado=palindromo( palabra)
        if(resultado):
            print(f" {palabra} es palíndromo")
        else:
            print(f" {palabra} NO es palíndromo")
    else:
        print("Uso python palindromo.py <palabra>")
if __name__ == "__main__":
    main()

```

- Sumar una lista de elementos, de forma recursiva.

```
import sys
def suma(lista:[int])>->int:
    #caso base
    if len(lista)==1 :
        return int(lista[0])
    #caso recursivo
    else:
        return int(lista[0])+ suma(lista[1:])
def main():
    #analisis de los parámetros de entrada
    if len(sys.argv) >= 2:
        try:
            lista = args = sys.argv[1:] # omite el primer elemento

            resultado=suma( lista)
            print(f"El resultado es: {resultado}")

        except ValueError:
            print(f"Error: algún parámetro no es entero.")

    else:
        print("Uso python factorial.py <n>")
if __name__ == "__main__":
    main()
```

- Resolver las torres de Hanoi.

Cada llamada hanoi(n, A, C, B):

- Mueve n-1 discos de A a B (usando C como auxiliar).
- Mueve el disco más grande de A a C.
- Mueve los n-1 discos de B a C (usando A como auxiliar).

```
import sys
def hanoi(n, origen, destino, auxiliar):
    if n == 1:
        print(f"Mover disco 1 de {origen} → {destino}")
        return

    # Mover n-1 discos desde origen a auxiliar
    hanoi(n - 1, origen, auxiliar, destino)
```

```

# Mover el disco más grande al destino
print(f"Mover disco {n} de {origen} → {destino}")

# Mover los n-1 discos del auxiliar al destino
hanoi(n - 1, auxiliar, destino, origen)

def main():
    #análisis de los parámetros de entrada
    if len(sys.argv) == 2:
        n = sys.argv[1]
        try:
            numero = int(n)
            hanoi( numero, "O", "D", "A")
        except ValueError:
            print(f"Error: '{n}' no es un número entero válido.")

    else:
        print("Uso python hanoi.py <n>")

if __name__ == "__main__":
    main()

```

- Dada una cadena, genera todas las combinaciones posibles de sus letras.

```

import sys
def combinaciones(palabra:str,actual:str):
    #caso base
    if len(palabra)==1 :
        print (f"{actual}{palabra}")
    #caso recursivo
    else:
        for i in range (len(palabra)):
            print(f"{actual}{palabra[i]}")
            nueva_palabra = palabra[:i] + palabra[i+1:]
            nuevo_actual=actual+palabra[i]
            combinaciones(nueva_palabra,nuevo_actual)

def main():
    #análisis de los parámetros de entrada
    if len(sys.argv) == 2:
        palabra = sys.argv[1]
        combinaciones( palabra,"")

    else:
        print("Uso python combinaciones.py <palabra>")

if __name__ == "__main__":

```

```
main()
```

7. Investigar e implementar el recorrido BFS y DFS usando networkx para el grafo del ejercicio 3 desde el nodo A.

8. Implementar las funciones para el algoritmo de Backtracking del ejercicio indicado en los apuntes :

- `es_solucion(elementos: list[int], solucion_parcial: list[bool], objetivo: int)-> bool`
- `siguiente_solucion(solucion_parcial:list[bool],siguiente:bool)->list[bool]`
- `es_valido(elementos:list[int], solucion_parcial:list[bool],objetivo:int)->bool`

```
# indica si es una solución valida
def es_solucion(elementos: list[int], solucion_parcial: list[bool], objetivo: int)-> bool:
    suma=0
    for i, valor in enumerate(solucion_parcial):
        if( solucion_parcial[i]):
            suma+= elementos[i]
    return suma==objetivo

#se añade a la solucion parcial el siguiente
def siguiente_solucion(solucion_parcial:list[bool],siguiente:bool)->list[bool]:
    #se copia y se añade
    siguiente_estado=solucion_parcial.copy()
    siguiente_estado.append(siguiente)

    return siguiente_estado

#indica si el estado es válido para seguir buscando una solución
def es_valido(elementos:list[int], solucion_parcial:list[bool],objetivo:int)->bool:
    suma=0
    for i, valor in enumerate(solucion_parcial):
        if( solucion_parcial[i]):
            suma+= elementos[i]
    #se pasa en este estado
    if suma>objetivo :

        return False

    #se suman todos los pendientes para ver si se llegaría al objetivo
    for i, valor in enumerate(elementos[len(solucion_parcial):], start=len(solucion_parcial)):
        suma+= valor
    #no va a llegar al objetivo
    if suma<objetivo :
        return False
    return True
```

9. El problema de la mochila es el clásico algoritmo que se resuelve entre otros por backtracking (existen otros métodos).

Se define así:

Dado un conjunto de n objetos, cada uno con un peso $w[i]$ y un valor $v[i]$, y una mochila que puede soportar un peso máximo W , determinar qué objetos incluir en la mochila de forma que:

- La suma total de los pesos no exceda la capacidad W .
- La suma total de los valores sea máxima.

Usar DFS o BFS (a mano, no usar librerías de terceros) y backtracking, comparando tiempos de ejecución para diferentes tamaños del problema.

Existen otras variantes con valores para cada elementos de forma que se maximice o minimice el coste, se opta por hacer la más sencilla, aunque solo sería necesario modificar la evaluación del valor de los resultados calculados. Pasando a ser un problema de optimización.

DFS:

```
def
mochilla(maximo:int,pesos:[int],seleccionados:[],peso_actual:[int])>-in
t: # type: ignore
    mejor_valores=seleccionados.copy()
    #caso base
    if(peso_actual>maximo):
        return None
    #para cada i se van añadiendo elementos
    for i in range(len(pesos)):
        nuevo_seleccionados=seleccionados.copy()
        #se añade el item a la seleccion
        nuevo_seleccionados[i]=nuevo_seleccionados[i]+1
        #se vuelve a calcular, si devuelve None es que se ha pasado el
peso
    resultado=mochilla(maximo,pesos,nuevo_seleccionados,peso_actual+pesos[i
])
    if(resultado!=None):
        #calcular el nuevo peso y ver si es mejor que el actual
        nuevo_peso=calcula_pesos(pesos,resultado)
        if(nuevo_peso>peso_actual):
            mejor_valores=resultado.copy()
            #peso_actual=nuevo_peso
    #devolver la mejor seleccion
    return mejor_valores
```

```

def calcula_pesos(pesos:[int],elementos:[int])>int:
    calculado=0
    for i in range(min(len(pesos),len(elementos))):
        calculado=calculado+pesos[i]*elementos[i]
    return calculado

def main():
    matriz=mochilla(9,[3,8,5,6],[0,0,0,0],0)
    print(matriz)

if __name__ == "__main__":
    main()

```

Backtracking:

```

def mochilla(indice:int,
            capacidad:int,
            pesos:[int],
            seleccionados:[int],
            mejor_seleccionado:[int],
            mejor:[int],
            )>[int,[ ]]: # type: ignore

    peso_actual=calcula_pesos(pesos,seleccionados)

    #caso base
    if(peso_actual>capacidad):

        return [mejor, mejor_seleccionado]
    #se tiene un buen valor,
    if peso_actual>mejor :
        mejor=calcula_pesos(pesos,seleccionados)
        mejor_seleccionado=seleccionados[:]
    #para cada i se van añadiendo elementos
    for i in range(indice,len(pesos)):
        if peso_actual + pesos[i] <= capacidad:
            seleccionados[i]=seleccionados[i]+1
            #se vuelve a calcular, si devuelve None es que se ha pasado
            el peso

    mejor,mejor_seleccionado=mochilla(i,capacidad,pesos,seleccionados,mejor
    _seleccionado,mejor)
    seleccionados[i]=seleccionados[i]-1

```



```

    return mejor,mejor_seleccionado

def calcula_pesos(pesos:[int],elementos:[int])>->int:
    calculado=0
    for i in range(min(len(pesos),len(elementos))):
        calculado=calculado+pesos[i]*elementos[i]
    return calculado

def main():
    matriz=mochilla(0,9,[3,8,5,6],[0,0,0,0],[0,0,0,0],0)
    print (f"Se ha llenado hasta {matriz[0]} con los pesos:
{matriz[1]}")
if __name__ == "__main__":
    main()

```

10. Dado el siguiente mapa, realizar una traza del algoritmo A* indicando el estado de las estructuras de datos.

